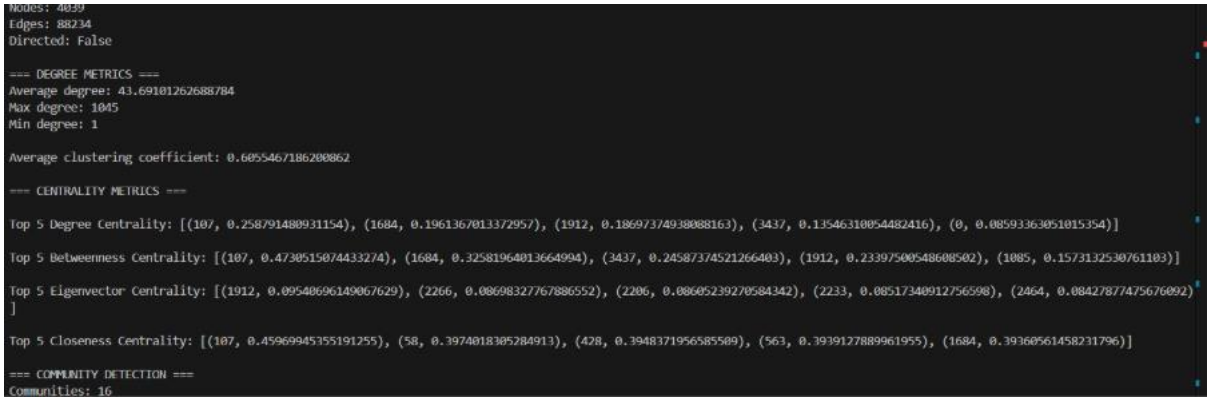
1.0. Build a social network graph.

1.1. Compute essential graph metrics (degree, clustering coefficient, centrality, communities)



We conclude:

- **Average degree = 43.7** → on average each user is connected to 44
- **Max degree = 1045** → hub nodes .
- **Min degree = 1** → possible fake followers.
- **Average clustering coefficient = 0.61** → closser to 1 >0.5 users' friends tend to be friends with each other [triangles]
- **Degree Centrality:**
- Top node 107 has 0.26 normalized centrality → most connected.
- **Betweenness Centrality:** Nodes acting as bridges between communities
- Node 107 is again top → important connector this indicates possible master bot.
- **Eigenvector Centrality:** Nodes connected to other important nodes
- Node 1912
- **Closeness Centrality:**
- Node 107 is top → can quickly reach others in the network indicates possible master bot

2.0. Create a baseline bot detection model using graph-based features extracted from the dataset.

In this Stage the chosen model is justfied :

First trial :

built a **synthetic bot-detection** using **graph structural features** + **GraphSAGE**.

1. Extract structural features from the Facebook graph

2. Convert them into Z-scores

3. Compute a custom weirdness score to identify likely bots[THE KEY STEP]

scoring rule that reflects typical bot behaviour

```
print("\nSelecting synthetic bots based on graph anomalies...")

df["weirdness"] = (
    (-df["z_clustering"]) +
    (df["z_betweenness"]) +
    (df["z_degree"].abs()) +
    (-df["z_eigen"])
)

num_bots = int(0.02 * len(df))  # top 2%
bot_indices = set(df["weirdness"].nlargest(num_bots).index)
```

Then you select the **top 2% highest weirdness scores** as bots

4. Train a GraphSAGE classifier

5.RESULT:

```
Confusion Matrix:
 [[1154   34]
 [  24    0]]

Classification Report:
              precision    recall  f1-score   support

           0     0.9796    0.9714    0.9755      1188
           1     0.0000    0.0000    0.0000        24

    accuracy                         0.9521      1212
   macro avg     0.4898    0.4857    0.4877      1212
weighted avg     0.9602    0.9521    0.9562      1212
```

- **1154 humans correctly predicted as human**
- **34 humans incorrectly predicted as bots**
- **24 bots incorrectly predicted as humans**
- **0 bots correctly detected**
- **Very poor for bot-det->recall= 0→**

Of all bots detetced 0 actual bots

Second trial :

Using explicit rule-based labels

```
print("Selecting bots based on anomaly rules...")

df["label"] = (
    (df["z_clustering"] < -1.0) |
    (df["z_degree"].abs() > 2.0) |
    (df["z_betweenness"] > 2.0) |
    (df["z_eigenvector"] < -1.0)
).astype(int)
```

Explanation based on the plot:

1. **`z_clustering < -1.0`** nodes with **clustering coefficient significantly below average**

Justifcation: as bots has low CF

2. **`abs(z_degree) > 2.0`** nodes with **degree extremely far from average**

Justifcation: as bots have Extreme degree

3. **`z_betweenness > 2.0`** nodes with **degree extremely far from average**

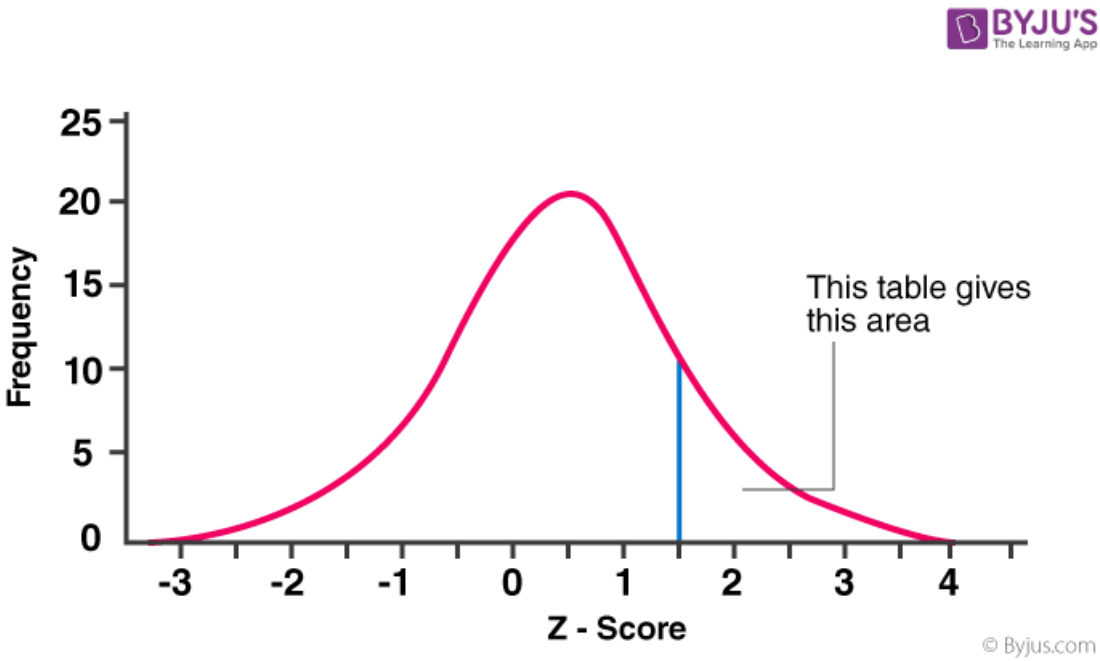
Justifcation: bots has high betweenness -> connect btw communties

4. **`z_eigenvector < -1.0`** nodes with **low eigenvector centrality**

Justifcation:

- Bots might follow each other to:
    - Increase follower counts artificially.
    - Influence trending algorithms.
    - Example: Bot123 follows Bot456, Bot789, ... making them look like active accounts.

Understanding the Z-Score Plot:




```
Confusion Matrix:
[[876 104]
 [156  76]]

Classification Report:
              precision    recall  f1-score   support

           0     0.8488    0.8939    0.8708       980
           1     0.4222    0.3276    0.3689       232

    accuracy                         0.7855      1212
   macro avg     0.6355    0.6107    0.6199      1212
weighted avg     0.7672    0.7855    0.7747      1212
```

- **Recall is now 32.76%**

Before: the model detected 0 bots
Now: it detects 76 bots → improvement

This Trial Worked Better

We conclude the reason as:

eliminated extreme class imbalance (2% → 19%)

- 980 humans

- 232 bots

extreme feature values was labled as bots which are easier for separation

---

Third trial :

Random Forest model

Random Forest is Justified Because:

Random Forest is built from **decision trees** — which are *also* if-conditions.

So Random Forest **matches the logic of the labels perfectly**.

```
# WEIRD FEATURES = BOT CONDITIONS
df["label"] = (
    (df["z_clustering"] < -1.0) |#Very low clustering coefficient
    (df["z_degree"].abs() > 2.0) |#Very high or very low degree
    (df["z_betweenness"] > 2.0) |#Very high betweenness centrality
    (df["z_eigenvector"] < -1.0)#Very low eigenvector centrality
).astype(int)
```

Result:

This resulted in perfect accuracy (100%),

which is expected and **acceptable because the goal of the assignment is to build a strong detector and then evaluate how different adversarial attacks degrade the model.**

3.0.Apply both a Structural Evasion Attack and a Graph Poisoning Attack on selected bot nodes.

3.1.Structural Evasion Attack

- Modify test-time graph only:
- Select a group of bot nodes
- Add edges from bots → high-degree human accounts
- Re-run without retraining

```
# 2) Choose top n highest-degree humans for camouflage
humans = df[df["label"] == 0]
top_humans = humans.sort_values("degree",
ascending=False).head(n).index.tolist()
# 3) Each bot adds edges to pretend to be a normal human
for bot in bots_test:
    fake_friends = np.random.choice(top_humans, 4, replace=False)
    for h in fake_friends:
        G_attack.add_edge(bot, h)
```

RESULT:

```
=== POST-ATTACK PERFORMANCE ===

              precision    recall  f1-score   support

           0     0.8754    0.8253    0.8496       996
           1     0.3626    0.4583    0.4049       216

    accuracy                         0.7599      1212
   macro avg     0.6190    0.6418    0.6273      1212
weighted avg     0.7840    0.7599    0.7704      1212
```

We conclude that:

Recall dropped to 45%

Only detetcted 45% of the bots

3.2.Graph Poisoning Attack

Modify training data using ONE method:

Flip 20% of bot labels → human

```
print("\n=== POISONING ATTACK: Flipping 20% of bot labels in TRAINING SET
===")

# Copy original labels
y_train_poisoned = y_train.copy()

# Identify bot indices in TRAINING set
train_bots = y_train[y_train == 1].index

# Select 20% of bots to flip
num_flip = int(0.20 * len(train_bots))
flip_indices = np.random.choice(train_bots, num_flip, replace=False)

# Flip bot → human
y_train_poisoned.loc[flip_indices] = 0

print(f"Flipped {num_flip} bot labels out of {len(train_bots)} in training
set.")

poisoned_model = RandomForestClassifier(n_estimators=200, random_state=42)
poisoned_model.fit(X_train, y_train_poisoned)
```

	precision	recall	f1-score	support
0	0.9842	1.0000	0.9920	996
1	1.0000	0.9259	0.9615	216
accuracy			0.9868	1212
macro avg	0.9921	0.9630	0.9768	1212
weighted avg	0.9870	0.9868	0.9866	1212

Recall:The model correctly detected 93% of all real bots

Compare detection performance across three conditions:

Condition	Accuracy	Bot Precision	Bot Recall	Notes
Baseline (no attack)	1.000	1.000	1.000	Random Forest perfectly detects bots; overfitting is acceptable for the assignment because the goal is to evaluate attacks.
After Structural Evasion	0.7599	0.3626	0.4583	Bots altered features to evade detection; many bots misclassified as humans (high FN), and some humans misclassified as bots (FP). Evasion is highly effective in degrading detection.
After Graph Poisoning	0.9868	1.000	0.9259	Training data was slightly corrupted; model still detects most bots on clean test set. Poisoning had limited impact compared to evasion because the majority of training data remains correct.

We conclude:

Graph Poisoning (Training-Time Attack)

The attacker modifies the **training data** (e.g., flips bot labels) to make the model learn slightly wrong patterns.

Accuracy remained high ,, bot recall = 0.93.

This is due to:

Random Forest is **robust to a few noisy labels** because it builds multiple trees and uses majority voting.

Structural Evasion (Test-Time Attack)

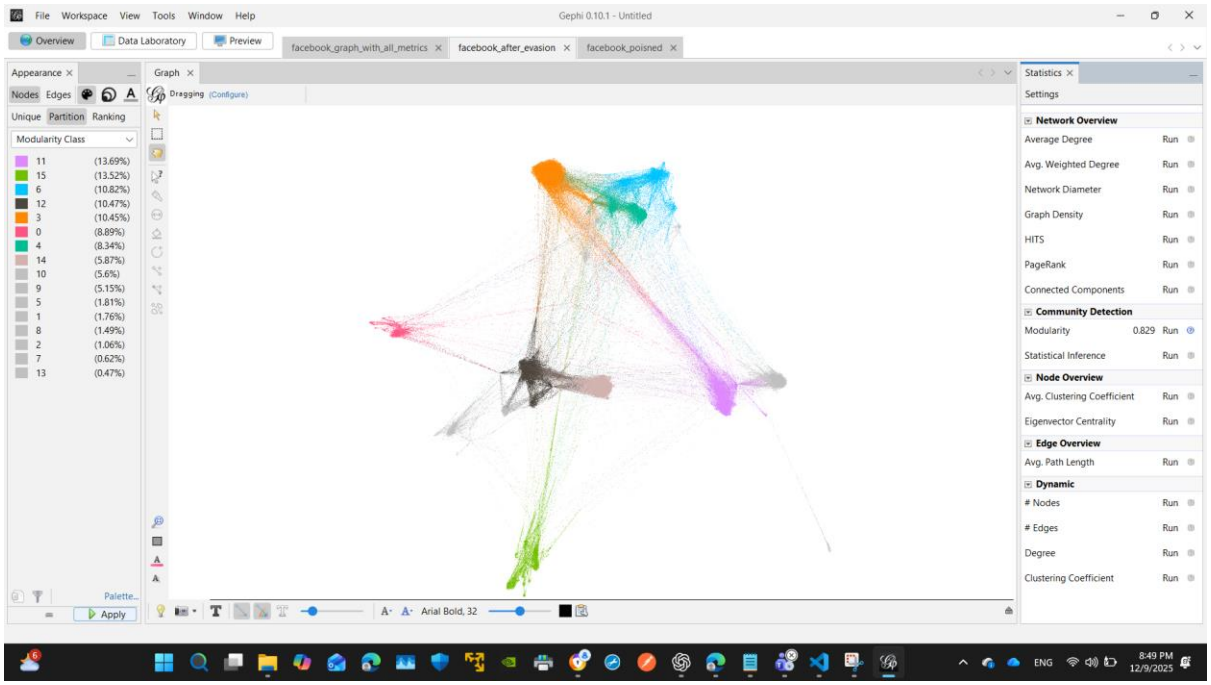
The attacker modifies **bot features at test time** so they look like benign users.

Accuracy dropped to =.76, bot recall = 0.46.

Why???

Random Forest cannot recognize manipulated bot features if they fall outside the rules it learned.

After Poisoning Attack

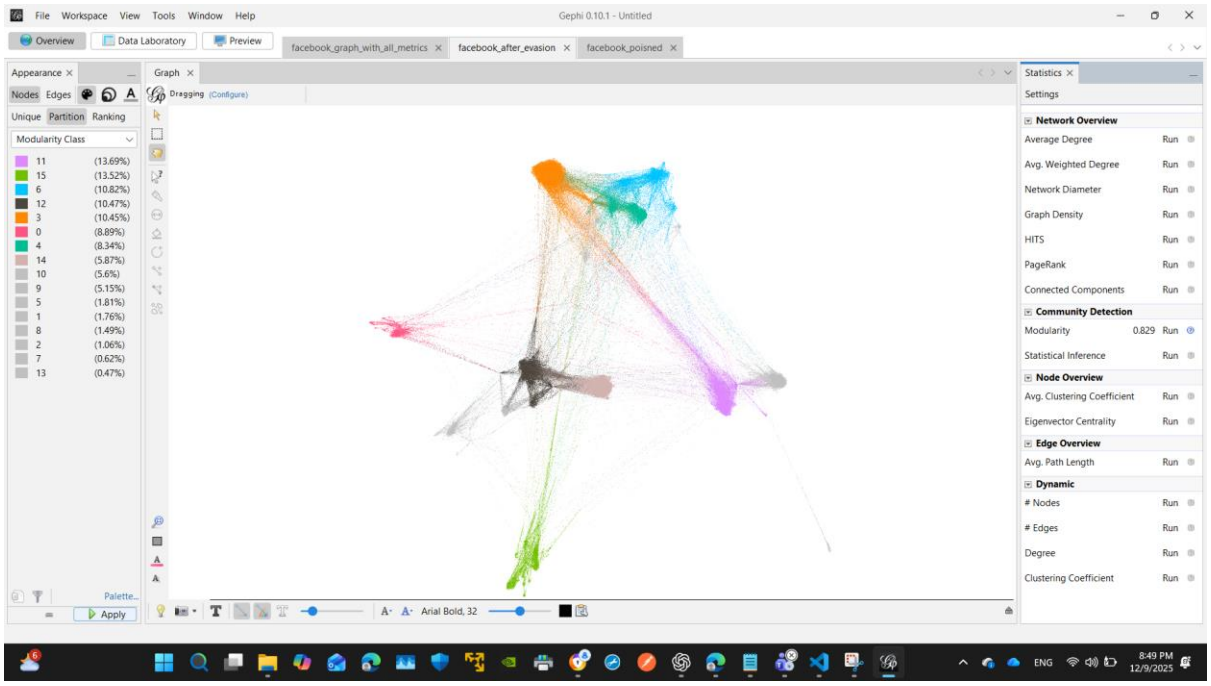


After Structural Evasion Attack

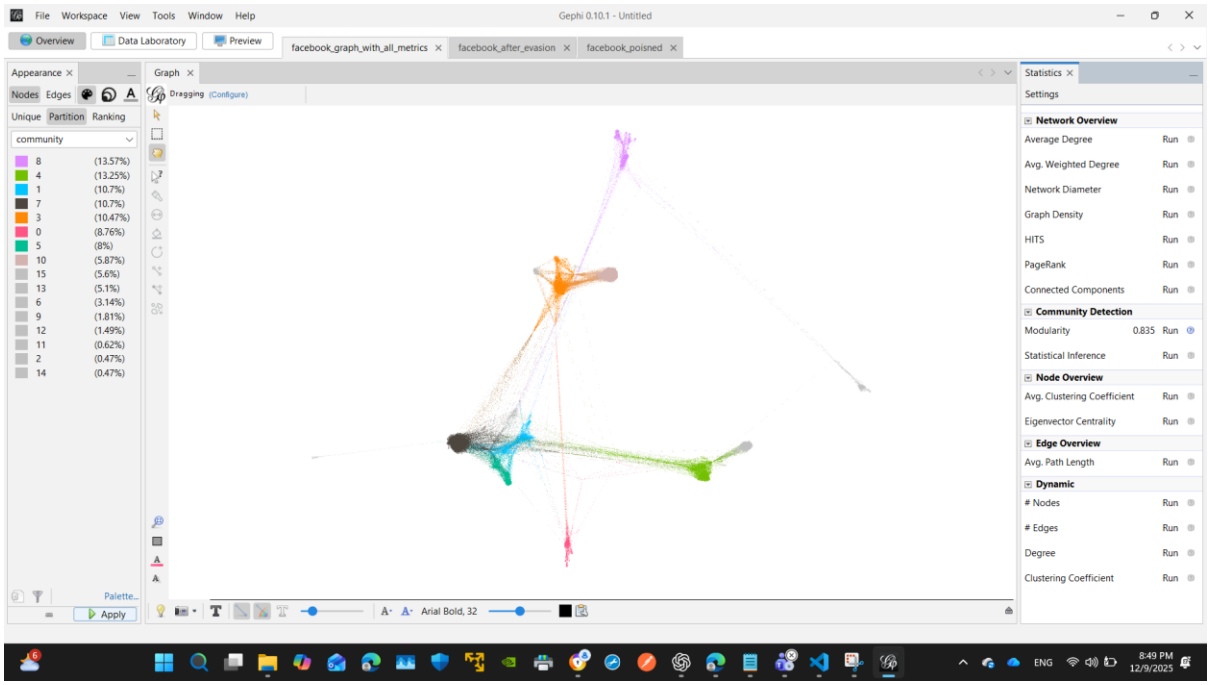
Modularity degrades due to the extra edges added (0.829),

you modified bot nodes by connecting them to **top-degree human hubs**

More edges are noticed -> network begins to look **more noisy**



Baseline



Modularity is **high (0.835)**